

研究背景

统计数据库管理系统广泛用于技术、金融、医疗等领域，为决策提供支持。随着隐私保护需求的增长，**差分隐私**机制成为统计数据库隐私分析的核心手段。差分隐私确保在查询结果中难以识别个体信息，其理论框架由 Dwork 等人提出。近年来，差分隐私技术已被部署于多个实际系统：美国人口普查局的 OnTheMap 项目、Apple 的本地差分隐私以及 Google Chrome 的设置分析等都采用了差分隐私方法。然而，差分隐私假设策展人（数据持有者）是可信的，在此模型中数据库以明文或可解密形式存储。这种假设在现实中日益受挑战：数据泄露、内部威胁和外部攻击事件频发，人们开始关注如何在不信任的环境中数据库本身进行安全保护。

之前研究的不足

传统的差分隐私实践依赖**可信策展人**模型，即假设数据库持有者会安全地保管原始数据。然而在实际应用场景（如云存储）中，这一假设并不总是成立。为此，一种称为**全隐私（pan-privacy）**的机制曾被提出：策展人仅保留一个带差分隐私的“状态”而不保存原始数据，即使该状态被泄露也难以恢复原始数据。尽管全隐私机制在理论上对一次性内侵提供了隐私保护，但它存在显著缺陷：一方面，该机制通常是有损的，原始数据库无法完整恢复；另一方面，全隐私只保证对无法进行交互式统计分析的攻击者提供隐私（如持久或快照攻击），而对能够多次查询的分析者却只能提供标准的差分隐私保护。更糟糕的是，许多全隐私机制在遭受多次攻击或持续泄露时性能急剧下降，失去了效用。此外，现有的加密数据库技术（如结构化加密、可搜索加密等）虽然能保护数据不被服务器窃取，但并未设计支持差分隐私查询。综上，这些不足表明：需要一种新的方案同时满足数据库的加密安全和统计查询的差分隐私需求。

核心创新点

本文针对上述问题提出了**私有加密数据库（Private Encrypted Database, PEDB）**的概念：即在结构化加密的基础上，引入差分隐私查询支持，实现对数据库的**加密存储与差分隐私统计查询**的双重保障。作者的主要创新包括：

- PEDB模型定义**：扩展传统的结构化加密模型，规定数据库应支持两类操作：由策展人执行的加密查询/更新操作和由分析者执行的差分隐私统计查询操作。前者保证数据在服务器端的加密安全，后者保证分析结果满足差分隐私。
- CPX（Encrypted Differential Privacy Counter）**：引入一种差分隐私加密计数器，作为可加密统计计数的数据结构。CPX 基于 Chan 等人的二元机制设计，并使用加法同态加密（AHE）实现。具体地，它维护一个范围树，每次更新时对叶到根路径的节点加密累加，并在特定节点注入拉普拉斯噪声；在查询时，服务器返回覆盖时间区间的节点加密和，解密后得到带噪计数。这种设计使得 CPX 的私密读取协议满足 ϵ -差分隐私。
- HPX（Encrypted Database for Histograms）**：构建基于结构化加密数据库和多个 CPX 计数器的方案，实现对频繁统计直方图查询的支持。HPX 使用底层的加密多映射（multi-map）存储加密数据库，同时为每个直方图桶维护一个 CPX 计数器。通过将数据库更新操作映射（Map1）到对应的计数器，HPX 在策展人更新数据库的同时对计数器进行增量更新，从而支持私密直方图查询。HPX 的整体设计充分考虑了新增的对手模型，包括**持久型对手**（持续监控交互过程的服务器）、**快照型对手**（获取数据库某时刻的副本）和**统计型对手**（只能看到查询输出的分析者）。

方案优势

本文方案在不信任服务器环境下实现了加密安全与差分隐私的兼顾：策展人存储在服务器的数据始终是加密的，防止数据泄露；而对分析者返回的统计结果则由差分隐私机制噪声保护。与传统依赖可信策展人的差分隐私不同，HPX 允许策展人将数据库托管给外部服务器（例如云服务）而无需暴露敏感信息。同时，HPX 能够支持**持续的查询和更新**：数据库可以动态地被策展人更新，而分析者可以随时提出私密

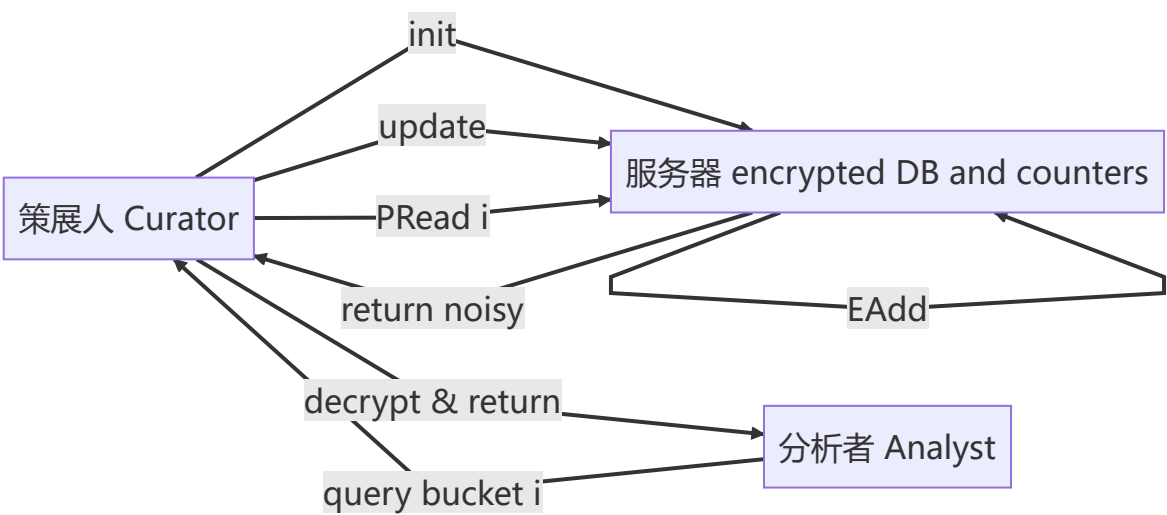
查询，每次统计仍满足差分隐私保证。更重要的是，HPX 针对多种攻击模型提供了正式的安全保证：对持久型对手或快照型对手，HPX 仅泄露底层结构化加密方案的最小泄露信息；对统计型对手，则直接由差分隐私机制提供保护。相比之下，以前的全隐私方案通常不可恢复数据且对多次入侵无力，而 HPX 在任何时刻都保留着可解密的原始数据库（仅在分析结果上附加噪声），因此可恢复性更好，并且能在多次查询场景下保持较高效用。

方案设计与实现流程

本文方案由 CPX 计数器和 HPX 加密数据库两部分构成。其总体流程可概述如下：

- **CPX 计数器设计：** CPX 内部采用**范围树**结构维护计数器值。策展人在初始化时为计数器生成公私钥对，服务器保存加密的树结构。每当有更新（加1、减1或0）时，服务器将该更新值用公钥加密，并将密文加到当前时间点对应的叶到根路径上。此外，为保证差分隐私，服务器在部分路径节点注入 Laplace 分布噪声（双曲范围为 $O(\log \lambda/\epsilon)$ ），这些噪声也以密文形式累加在节点中。当需要读取当前计数时，服务器选取覆盖区间 $[1, t]$ 的最小节点集合，将这些节点上的密文相加发送给解密者。解密后即可得到当前计数的真实值加上噪声，满足 ϵ -差分隐私。
- **HPX 加密数据库设计与流程：** HPX 由一个加密的多映射结构（Encrypted DB）和若干 CPX 计数器组成。设数据库记录更新操作集为 U ，定义函数 $Map1(u)$ 将更新操作 u 映射到一个或多个直方图桶索引。系统流程：
 1. **初始化：** 策展人运行底层结构化加密方案 $\Sigma_{DB}.Setup$ 将数据库以密文形式存储到服务器上，并为每个直方图桶初始化一个CPX计数器，服务器保存所有加密结构。
 2. **数据更新（加/删记录）：** 策展人对数据库执行加密的更新操作 $\Sigma_{DB}.EAdd$ 或 $ERemove$ ，同时告知服务器此操作对应的直方图桶索引 $i \in Map1(u)$ 。服务器首先在加密数据库中更新数据，然后对**每个** CPX 计数器执行 $EAdd$ ：如果计数器索引 $i \in Map1(u)$ 则加 1（或减 1），否则加 0，以**隐藏真正发生更新的桶编号**。因此服务器对所有计数器进行操作，但仅目标桶会发生有效变化，其他桶的“更新”值为 0。
 3. **私密查询（直方图查询）：** 分析者针对某个桶 i 发起查询。服务器执行对应计数器的 $PRead$ 协议：将覆盖 $[1, t]$ 的节点密文累加并发送给策展人。策展人利用私钥解密，得到该桶的真实计数值加噪声，并将结果返回给分析者。

上述流程可以通过下图流程图示意：



其中 CPX 计数器与加密数据库协同工作，实现了对直方图查询的保护：服务器始终只能看到加密数据和密文计数器，不学习具体内容，分析者只能收到带噪结果，从而同时满足加密安全和差分隐私要求。

主要结果与结论

本文在理论和实验上证明了方案的有效性和安全性：

- 效率：** CPX 的加密更新和私密读取操作时间复杂度均为 $O(\log \lambda \cdot T_{AHE})$ (λ 为更新总次数, T_{AHE} 为同态加密运算时间)。HPX 的查询复杂度等于底层结构化加密的查询复杂度, 更新操作的开销等同于底层加密更新加上对每个桶的一个CPX更新 (桶数量线性因子)。实际性能评估显示: 在使用2048位Paillier同态加密和一种高效多映射加密方案(DLS)的设置下, 对 1000 万条记录、25 个直方图桶的数据集进行实验, HPX.Setup 约需 10 分钟, 单次更新 (增加/删除记录) 约0.96分钟, 而单次查询 (EQuery+PRead) 约需1微秒。总体而言, 方案具有可接受的性能, 尤其查询开销极低。
- 差分隐私保证：** 理论上证明了 CPX 的 *PRead* 协议满足 ϵ -差分隐私。进一步, HPX对于直方图查询满足 $2\epsilon \cdot \max_{u \in U} |Map1(u)|$ -差分隐私 (Theorem 6.5), 其中 $\max |Map1(u)|$ 是单次更新可能影响的最大桶数。也就是说, HPX 输出的隐私保护级别与底层 CPX 计数器同阶。
- 安全性：** 在自定义的**适应性持久安全**和**快照安全**模型下证明, HPX 方案满足严格的安全性定义。也就是说, 对于持续监控服务器或获得服务器快照的攻击者, 只能通过预先定义的泄露函数 (如数据大小、查询模式) 模拟真实交互, 无法获得额外信息。此外, 如果底层加密方案本身对串改攻击是抗击穿的 (breach-resistant), HPX 也继承了这一性质。
- 实用性估算：** 实验结果表明, 在保证 ϵ -差分隐私的同时, 方案对大规模统计查询具有实用性。性能测试和对抗模型证明表明, HPX能够在有效防御多种攻击的同时, 提供可恢复的加密存储和可用的统计分析。

延伸思考问题

- 如何扩展到其他统计查询？** 本文仅针对直方图查询设计 HPX。能否推广到其他类型的统计查询 (如线性查询、频率矩阵、联结查询等)? 例如, 对加法计数器的设计是否需要调整, 或者是否需要引入新的加密数据结构来支持不同查询?
- 系统集成与优化：** 在系统层面, CPX/HPX 能否与已有的加密数据库 (如 CryptDB、Monomi 等) 或差分隐私系统 (如 PINQ、GUPT) 结合? 如何在实际系统中无缝集成私密加密查询, 兼顾查询性能和安全性? 此外, 对于不同应用场景, 是否可以采用更高效的同态加密方案或数据结构来优化 CPX?
- 实际部署挑战：** 如果在云服务或分布式环境中部署, HPX 如何处理实时数据更新、并发查询以及资源开销问题? 面对海量数据和高频率更新时, 如何设计高效的更新策略和噪声分配机制? 在多租户或分层信任模型下, HPX 又能否扩展以满足更复杂的安全需求?

1. 差分隐私的基本保证

- 差分隐私 (Differential Privacy)** 保证: 对于任何两个“相邻”数据库 (仅相差某一条记录), 在对它们分别执行某个查询机制后, 得到某个答案的概率之比 **不超过** e^{ϵ} 。
- 直观上就是: 无论一个人是否在数据库中, 其对最终查询结果的影响都被噪声“掩盖”了。

这一定义本身并不区分“对手怎么拿到查询结果”——只要结果符合上面那个概率界, 它就是 ϵ -差分隐私。

2. “能发起查询的对手”（分析者 adversary）

- 他们拥有**查询接口**，可以多次向系统提问（例如：“有多少人打过疫苗？”、“有多少人年龄在20-30岁？”）
- 系统每次回答都会加上噪声，以满足 ϵ -DP。

对于这类对手，系统给出的“差分隐私答案”已经是**最强**的隐私保证：你无法在统计上分辨任意一个人的存在与否，不管对手怎样提问，也只能拿到加噪声的结果。

3. “偷到状态的对手”（非分析者 adversary）

- 他们**没有查询接口**，只能一次性拿到系统内部维护的“状态”——例如某个计数器或数据摘要，此状态里也带了噪声。
- 拿到这个状态后，他们**不能**继续问更多问题，只能静态地分析这份有噪声的快照。

全隐私机制就是专门设计来保护这种对手的：即使对手窃取了你内部状态，由于状态本身已经满足一次性差分隐私，他们同样**无法**恢复出精确数据，也无法判断某个人在不在数据库里。

4. 为什么说“对能发起查询的对手未加强保护”？

- 对于**能发起查询的对手**，全隐私机制**依旧**使用常规的差分隐私查询接口——每次回答时在真实答案上加噪声，满足 ϵ -DP。
- 也就是说，全隐私**并没有**引入任何新手段来让这个查询接口比普通差分隐私更“安全”或者更“抗攻击”。攻击者依然是拿到“噪声答案”——和不使用全隐私的场景没区别。

换句话说：

- **普通差分隐私系统**：只关心“对谁发查询”这件事，给出的噪声答案保证 ϵ -DP。
- **全隐私机制**：多了一层“状态快照也带噪声”的保护，但查询答案仍是那套老噪声方案，不会更严。

5. 举例帮助理解

1. 普通差分隐私

- 对手 A 每次查询都会加噪声，系统保证 ϵ -DP。
- 如果对手没有查询接口（比如他只是拿到了一张打印的查询日志），他没有别的途径。

2. 全隐私机制

- 系统内部额外维护一个带噪“状态”并定期存储。
- 对手 B 偷到这个状态文件，文件本身带噪声——他**不能**从里头恢复数据。
- **但如果**对手 C 同时既能偷文件，又能继续发起查询，那么：
 - 他拿到文件的那一刻，被保护得和普通DP查询一样。
 - 然后他继续发问时，得到的依然是普通DP噪声答案。

因此，全隐私虽然在“偷文件”场景下给了保护，但对“发查询”这种主流攻击模式，**并没有**额外提升对噪声策略或隐私预算的管理，他只能拿到和标准差分隐私系统一模一样的结果。

核心结论

- **全隐私：**
 - 强化了“状态被泄露时”的安全性。
 - **却不改变**“查询被调用时”的隐私强度——查询接口依旧是标准 ϵ -DP。
- 理想中，我们希望：
 - “偷状态”和“发查询”两种场景下，**都能**得到同样甚至更强的隐私保障。
 - 但现有全隐私机制只在“偷状态”场景加分，对查询场景毫无附加保护。

私有加密数据库

除了支持加密查询外，还支持差分隐私查询，即管理员使用加密查询来查询和修改数据，而分析师使用差分隐私查询来分析数据。

新的攻击者模型

在文章的环境中，有多个对手需要考虑，包括**存储数据库的服务器**、**分析数据的分析师**，以及可能在某个时间点**破坏服务器**并获取数据库快照的**对手**。

1. **持久性对手 (persistent adversary)**：能够永久性的入侵服务器，并能持续监视**管理者**、**分析师**和**服务器**之间的所有交互记录。
2. **统计对手 (statistical adversary)**：只能访问差分隐私查询的输出结果，不一定能看到完整的交互记录。对应差分隐私中不受信任的分析者，试图从查询中推断数据的敏感信息。
3. **快照对手 (snapshot adversary)**：多次访问服务器上存储的数据库静态副本，但无法获取任何交互记录的通信内容。

和传统的差分隐私相比，数据被存储在不受信任的服务器上，而且还会被不断修改。

针对隐私直方图查询的加密数据库

直方图是统计分析中最常见且核心的查询之一，许多其他重要查询（如列联表、边际分布等）都可以表示为直方图

解决这个问题的朴素方法是管理者使用结构化加密生成一个 EDB，服务器用全隐私直方图机制构建一个全隐私状态。不过这种方法只能保证针对持久和快照对手的差异隐私。

1. CPX：使用加法同态和差分私有计数器来构建支持加密加操作和差分隐私读操作。
2. 将一个标准的加密数据结构与多个 CPX 加密计数器实例相结合，构建出一个支持私有直方图查询的隐私加密数据库。

HPX：避免加密数据库的操作和隐私计数器之间的交互泄露信息，影响 CPX 计数器的安全性提出的方案。

我们分别用 Q 、 R 和 U 表示数据对象的查询、响应和更新空间。

结构化加密的定义

互动式的结构化加密方案包含一个多项式时间算法和三个互动式的协议：

- $(st, K, \text{EDS}) \leftarrow \text{Setup}(1^k, \text{DS})$
- $(st', r; \perp) \leftarrow \text{Query}_{C,S}(st, K, q; \text{EDS})$

响应 r

- $(st'; \text{EDS}') \leftarrow \text{Add}_{C,S}(st, K, u^+; \text{EDS})$
- $(st'; \text{EDS}') \leftarrow \text{Remove}_{C,S}(st, K, u^-; \text{EDS})$

考虑其安全性

定义初始化、更新和查询的泄露函数 $\mathcal{L}$ ，定义交互式 STE 的自适应安全性为攻击者分不出由泄露函数组成的 **Ideal** 情况和真实场景组成的 **Real** 情况。

带差分隐私的结构化加密

由五个多项式时间协议组成：

- $((st, K_C); \text{PEDB}; K_A) \leftarrow \text{Setup}_{C,S,A}((1^k, \epsilon, \text{DB}); \perp; \perp)$
- $(st'; \text{PEDB}') \leftarrow \text{EAdd}_{C,S}((st, K_C, u^+); \text{PEDB})$
- $(st'; \text{PEDB}') \leftarrow \text{ERemove}_{C,S}((st, K_C, u^-); \text{PEDB})$
- $((st', r); \perp) \leftarrow \text{EQuery}_{C,S}((st, K_C, q); \text{PEDB})$
- $(r^p; \perp) \leftarrow \text{PQuery}_{A,S}((K_A, q^p); \text{PEDB})$

正确性和效用

正确性：从 PEDB_i 中搜索的结果以几乎确定（忽略可忽略概率）的概率返回正确响应，其中， PEDB_i 是在 Setup 算法产生的 PEDB_0 数据库上应用多次更新操作得来的数据库

效用：差分隐私搜索的结果和正确结果的差别相差不超过 α ，且这种准确度在概率至少在 $1 - \delta$ 的情况下成立。

安全性和隐私

差分隐私下的安全性考虑：要考虑的以下几个模型：

1. 持久性对手：可以访问加密数据库，并可以监视加密和私有查询操作。
2. 快照对手：它可以在入侵时访问加密数据库的副本，但看不到加密或私有查询操作。
3. 统计对手：它可以看到对隐私查询的响应，但看不到加密的隐私数据库或加密的查询/更新操作。

同时还要防止这几个对手之间相互勾结，比如

1. 快照对手与统计对手勾结。
2. 持久性对手和统计对手勾结。

此处的“Utility”（效用）定义了给定安全参数 k 下，一个带差分隐私的结构化加密方案 Δ 对私有查询返回结果的准确度保证。逐句拆解如下：

1. 方案结构

$\Delta = (\text{Setup}, \text{EAdd}, \text{ERemove}, \text{EQQuery}, \text{PQuery})$

- Setup: 初始化并生成加密数据库 PEDB_0 。
- EAdd、ERemove: 对加密数据库做增、删操作。
- EQQuery: 普通的加密查询接口 (供 curator 使用)。
- PQuery: 差分私有查询接口 (供 analyst 使用), 会在回答中加入随机噪声。

2. 多轮操作序列 σ

- $\sigma = (\sigma_1, \dots, \sigma_\lambda)$ 是长度 λ (多项式大小) 的操作序列, 包含若干更新操作 (增删) 和若干私有查询操作 $\sigma_i = q_i^p$ 。
- 每做一次更新, 方案就会把对应的加密数据结构从 PEDB_{i-1} 更新到 PEDB_i 。

3. 真正答案 r_i 与带噪声答案 r_i^p

- 在第 i 步, 如果操作 σ_i 是一个私有查询 q_i^p , 则执行 $(r_i^p; \perp) \leftarrow \text{PQuery}((K_A, q_i^p); \text{PEDB}_{i-1})$ 。
- 其中 r_i 表示查询 q_i^p 在未加噪声、基于真实明文数据库时应得的正确答案。

4. (α, δ) -useful 的含义

- 对于任意符合上述条件的多轮序列以及任何一次私有查询 i , 都有

$$\Pr[|r_i^p - r_i| \leq \alpha] \geq 1 - \delta.$$

- 换言之, 带噪声的答案 r_i^p 与正确答案 r_i 之间的差距, 至多是一个“加性误差” α , 并且以概率至少 $1 - \delta$ 成立。

5. “additive factor of α ”

- 这里的“加性因子 α ”就是指 $|r_i^p - r_i| \leq \alpha$ 中那部分: 允许噪声答案在真实值的基础上上下浮动至多 α 个单位。

总结: 一个 PSTE 方案如果是 (α, δ) -useful, 就意味着它对每一次差分私有查询都能保证——带噪声的查询结果与真实结果相差不超过 α , 且这种准确度在概率至少 $1 - \delta$ 的情况下成立。这样, 我们就能在差分隐私保护强度 (由噪声大小、隐私参数决定) 和统计结果精确度 (由 α, δ 控制) 之间取得平衡。

针对持久性对手的安全性

攻击者作为服务器, 分不清他在和管理员或分析师交互还是和两个不同的模拟器交互。

针对快照对手的安全性

1. 真实世界实验:

- 对手看到当前加密库 PEDB_{i-1} , 输出一批多项式长度的操作序列 $\sigma_i = (\sigma_{i,1}, \dots, \sigma_{i,m})$, 其中既包括更新 (EAdd/ERemove), 也包括私有查询 (PQuery)。
- 挑战者依次把 σ_i 中的操作应用到 PEDB_{i-1} , 得到新的 PEDB_i 。
- 对手接着看到 PEDB_i , 再输出下一批操作 σ_{i+1} 。

2. 理想世界实验:

- 对手见到模拟出的 PEDB_{i-1} , 输出操作序列 σ_i 。

挑战者在明文数据库上执行 σ_i , 由 DB_{i-1} 变为 DB_i 。

模拟器 S 从泄露函数 $\mathcal{L}_{SN}(DB_i, \sigma_i)$ 中获取允许泄露的信息, 再模拟出 $PEDB_{i-1}$ 。

即便对手多次、有节奏地获取数据库的加密快照, 也只能从中看到预先定义好的少量泄露信息 $\mathcal{L}_{SN}$, 而对底层明文数据及操作序列没有额外知识。

抗泄露

主要针对“**数据泄露**”场景——即对手反复盗取数据库快照, 却只能看到最少的元信息, 不得窥见任何实际内容。

接续上文的快照安全和自适应持久安全, 这里引入了“**抗泄露**” (breach-resistance) 的更强要求, 主要针对“**数据泄露**”场景——即对手反复盗取数据库快照, 却只能看到最少的元信息, 不得窥见任何实际内容。

1. 快照泄露只含两项

- **数据库规模** $|DB_i|$
- **私有查询空间规模** $|\mathcal{P}|$

2. 不泄露其它任何信息

- 包括具体的分桶分布、历史更新次数、查询模式等。
- 对手即便拥有任意频度的快照, 也只能得知这两个数值。

3. 意义

- 在实际数据泄露、设备被盗、法院传票等场景下, 系统泄露给对手的最坏信息仅是“你这里存了多大规模的数据, 以及允许分析者能问多少个不同的问题”, 而无任何业务或统计秘密。
- 这样就将泄露风险降到最低, 只向对手公开最“无害”的元信息。

通过这个定义, 论文保证在极端的“泄露”场景下, 敏感数据与差分私有查询的细节依然受到严格保护。

针对统计对手的安全性

非正式地说, 差分隐私通过要求对任意两条“相邻”数据库, 其机制输出的概率分布大致相同, 来形式化隐私。

这里的符号表示如下含义:

- $\text{Im}(M)$: 机制 M 的像 (image), 也就是它所有可能输出的集合。
- $\text{Im}(M)^\lambda$: 表示长度为 λ 的输出序列空间, 即所有形如 $(y_1, y_2, \dots, y_\lambda)$ 且每个 $y_i \in \text{Im}(M)$ 的元组所组成的集合。
- $S \subseteq \text{Im}(M)^\lambda$: 意味着 S 是这些长度 λ 输出序列中的一个子集。换句话说, **S 描述了“感兴趣”的那些输出序列模式。**

在持续观测下差分隐私的定义里,

$$\Pr[(M(DB_1), \dots, M(DB_\lambda)) \in S]$$

就是说“机制 M 在各时刻的 λ 次输出序列落在集合 S 里的概率”。

这保证: 不管管理者在某一步做或不做那一次“微小的”增删操作, 分析者看到的所有私有查询输出的联合分布, 都相差至多一个指数因子 e^ϵ , 即隐私风险受到控制。

差分隐私的定义:

$$\Pr \left[\left((r_{1,q^p}^{q^p})_{q^p \in \mathbb{P}}, \dots, (r_{\lambda,q^p}^{q^p})_{q^p \in \mathbb{P}} \right) \in S \right] \leq e^\epsilon \cdot \Pr \left[\left((r_{1,q^p}'^{q^p})_{q^p \in \mathbb{P}}, \dots, (r_{\lambda,q^p}'^{q^p})_{q^p \in \mathbb{P}} \right) \in S \right],$$

这个不等式是差分隐私的核心数学表达，直观上它说：

“无论被观测的整条私有查询结果序列落在什么事件 S 上，只要两个操作序列 σ 和 σ' 在至多一次更新上不同，那么对应的概率之比最多相差一个 e^ϵ 因子。”

拆开来说：

1. 左边的概率

$$\Pr \left[(r_{1,\cdot}, \dots, r_{\lambda,\cdot}) \in S \right]$$

这是在操作序列 σ 情况下，分析者对所有可能的私有查询在每个时刻都得到了答案 $\{r_{i,q^p}\}$ ， $i = 1 \dots \lambda$ ， $q^p \in \mathcal{P}$ ，并且这些答案整体落入某个“可疑事件”集合 S 的概率。

2. 右边的概率

$$\Pr \left[(r_{1,\cdot}', \dots, r_{\lambda,\cdot}') \in S \right]$$

这是在与之**相邻**的操作序列 σ' （仅有一次更新不同）下，分析者观察到对应答案序列落入同一事件 S 的概率。

3. e^ϵ 放缩因子

差分隐私要求这两个概率比值满足

$$\Pr[\sigma \text{ 下落入 } S] \leq e^\epsilon \Pr[\sigma' \text{ 下落入 } S].$$

- 当 ϵ 很小时， $e^\epsilon \approx 1$ ，说明改变一次更新对最终观测结果的概率影响非常小，几乎看不出差别。
- 这样就保证：即便对手知道输出的完整序列，也难以判断到底是否执行过那次“微小”更新，从而保护了更新中涉及的那条记录的隐私。

举个简单例子

- 假设 $\lambda = 1$ ，只有一次更新后只做一次查询，总共有两个可能的操作序列：
 - σ ：在数据库里加了一条敏感记录；
 - σ' ：没加这条记录。
- 查询的私有机制会在真实计数上加噪声，输出一个数字 r 。
- 如果这个机制满足 ϵ -差分隐私，那么对于任意我们关心的输出区间 $S = [a, b]$ ，都有

$$\Pr[r \in [a, b] \mid \sigma] \leq e^\epsilon \Pr[r \in [a, b] \mid \sigma'].$$

- 也就是说，“加入敏感记录”这件事并不会显著提高输出落在某个范围的概率。

将这个思想推广到**多次更新 + 多次查询、全体可能查询**的场景，就得到了你这条不等式的形式化表达。它保证了“整个输出序列”对那一次更新的敏感度被严格限制在 e^ϵ 的级别之内。

CPX：一种差分隐私加密计数器

计数器是一种支持 Add 和 Read 操作的数据结构。差分隐私加密计数器能够支持差分隐私 Read 和加密 Add 操作，通常通过加噪声来实现。

计数器表示为具有本地寄存器的范围树。本地寄存器只存储到目前为止执行的加法操作的总数。

范围树：



1. Setup

1. 数据库加密初始化

- 管理者 (Curator) 使用结构化加密方案对明文数据库进行 Setup, 生成并保存加密数据库结构和本地状态。

2. 直方图计数器初始化

- 针对直方图的每个桶, 管理者分别运行私有计数器的 Setup, 得到三份信息:
 - 一份由管理者保存的子密钥
 - 一份由服务器存储的加密计数器
 - 一份由分析者保存的子密钥

3. 分发角色数据

- 管理者保留: 数据库加密密钥、本地状态、所有计数器的管理者子密钥
- 服务器保留: 加密数据库结构、所有加密计数器
- 分析者保留: 所有计数器的分析者子密钥

2. EAdd (向数据库和直方图添加记录)

1. 数据库更新

- 管理者与服务器协同, 将明文更新 (插入) 应用到加密数据库结构并更新本地状态。

2. 更新确认

- 服务器检查加密数据库结构是否发生变化, 并将“成功/失败”标志回传给管理者。

3. 直方图计数器更新

- 管理者与服务器对每个桶都执行一次加密加法操作:
 - 对于真正被此次插入命中的桶, 加密加+1。
 - 其余桶, 加密加0。

4. 状态同步

- 管理者和服务器各自更新本地状态与存储结构, 完成一次增量操作。

3. ERemove (从数据库和直方图删除记录)

1. 数据库删除

- 管理者与服务器协同, 将明文删除操作应用到加密数据库结构并更新本地状态。

2. 删除确认

- 服务器检查加密数据库结构是否发生变化, 并回传“成功/失败”标志。

3. 直方图计数器更新

- 管理者与服务器对每个桶都执行一次加密加法操作:
 - 对于真正被此次删除命中的桶, 加密加-1。
 - 其余桶, 加密加0。

4. 状态同步

- 管理者和服务器更新各自状态与存储，完成一次删除操作。

4. EQuery（对加密数据库执行查询）

1. 发起查询

- 管理者与服务器按结构化加密协议交互，提交查询请求。

2. 返回结果

- 服务器执行查询并将（加密）中间步骤保密，最终只把明文结果返回给管理者。

3. 更新状态

- 管理者更新本地状态（若有）以便后续继续交互。

5. PHist（分析者的差分隐私直方图查询）

1. 选择桶编号

- 分析者指定想要查询的一个或多个桶编号。

2. 私有读取

- 对每个指定桶，分析者与服务器运行私有计数器的 PRead 协议，从相应加密计数器中读取带噪声的计数值。

3. 汇总输出

- 分析者收集所有带噪声计数，形成最终的私有直方图回答。

这样，HPX 通过这五个步骤模块化地结合了结构化加密与私有计数器，实现了对明文数据库的安全加密管理和对直方图查询的差分隐私保护。

直方图查询，本质上就是统计一组“桶（bin）”里各自有多少条记录。打个简单的比方：

- **数据集**：假设你有一堆用户注册信息，每条里都记录了用户的年龄。
- **直方图的定义**：你先把年龄分段，比如 0-17 岁、18-25 岁、26-35 岁、36-50 岁、51 岁以上，一共五个桶。
- **直方图查询**：你希望知道“每个年龄段里有多少用户”，结果就是一个长度为 5 的向量，比如 [120,450,780,320,90]。

在这篇文章里，直方图查询的作用就是让分析者（analyst）在**不直接看到明文数据库**的情况下，得到类似这样的“分类计数”结果：

1. **安全加密**：数据库和计数器都在服务器端以加密形式存储。
2. **隐私保护**：给出的每个桶的计数都被加上了噪声，满足差分隐私（DP），防止通过多次查询还原出某个用户的记录。

3. **动态更新**：当管理者往数据库里插入或删除记录时，相应的桶计数“加密加法”也会被同步更新，保证后续查询依然正确（尽管带噪）。

换句话说，直方图查询就是让你能够回答“各类别/分组的大小分布”这类最常见、最基础的统计问题——同时结合结构化加密和差分隐私，做到：

- **服务器看不到你的查询意图**（每次对每个桶都做一次同态加零），
- **分析者看不到具体哪个桶变动了多少**（噪声混淆了真实值），
- **却最终能得到一份总体的、带噪声的直方图。**

它在统计学里是离散分布最直观的表达，几乎所有更复杂的统计（交叉表、边际分布、多维直方图）都可以还原成对若干“桶”计数的查询。